

1. Purpose

This document records what currently exists in the repository as implemented by Layan Salem for the midterm, independent of any planned changes. Its purpose is to establish ground truth before the Research Plan (Step 3) defines what will be modified and compared against a baseline.

2. Module Inventory

The design consists of six RTL modules instantiated under a single top-level wrapper, `adaptive_pg_top.v` (~400 lines total):

Module	Role	Lines
<code>alu_32bit.v</code>	9-operation 32-bit ALU; the power-managed functional block	~70
<code>activity_monitor.v</code>	16-cycle rolling-window popcount; produces <code>idle_count</code> and <code>activity_rate</code>	~70
<code>hysteresis_predictor.v</code>	Dual-condition adaptive decision logic (the novel contribution)	~110
<code>power_state_fsm.v</code>	7-state FSM mediating <code>sleep_req</code> into actual power-mode transitions with wake-up delays	~150
<code>sleep_controller.v</code>	Footer gate-bias sequencing and two-phase rush-current control	~90

isolation_ctrl.v	Behavioral isolation model (ISO_AND equivalent) for simulation	~25
----------------------------------	--	-----

Supporting files: [upf/adaptive_pg.upf](#) (power intent: 2 domains, 1 power switch, isolation + retention strategy), [tb/tb_adaptive_pg_top.v](#) (5-workload testbench, ~280 lines), [scripts/dc_synthesis.tcl](#) and [scripts/icc_place_route.tcl](#) (Synopsys flow scripts).

3. Architecture

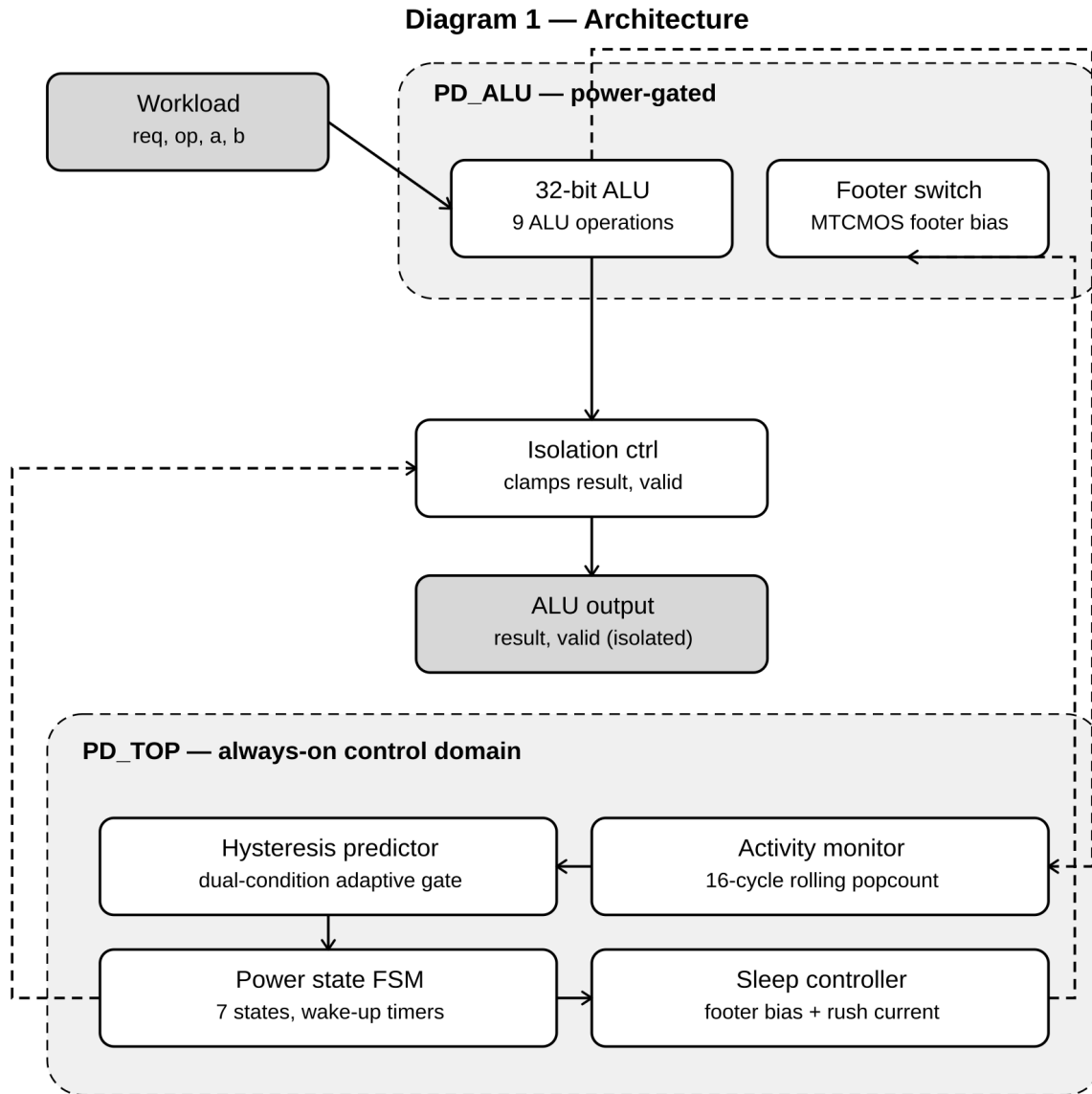
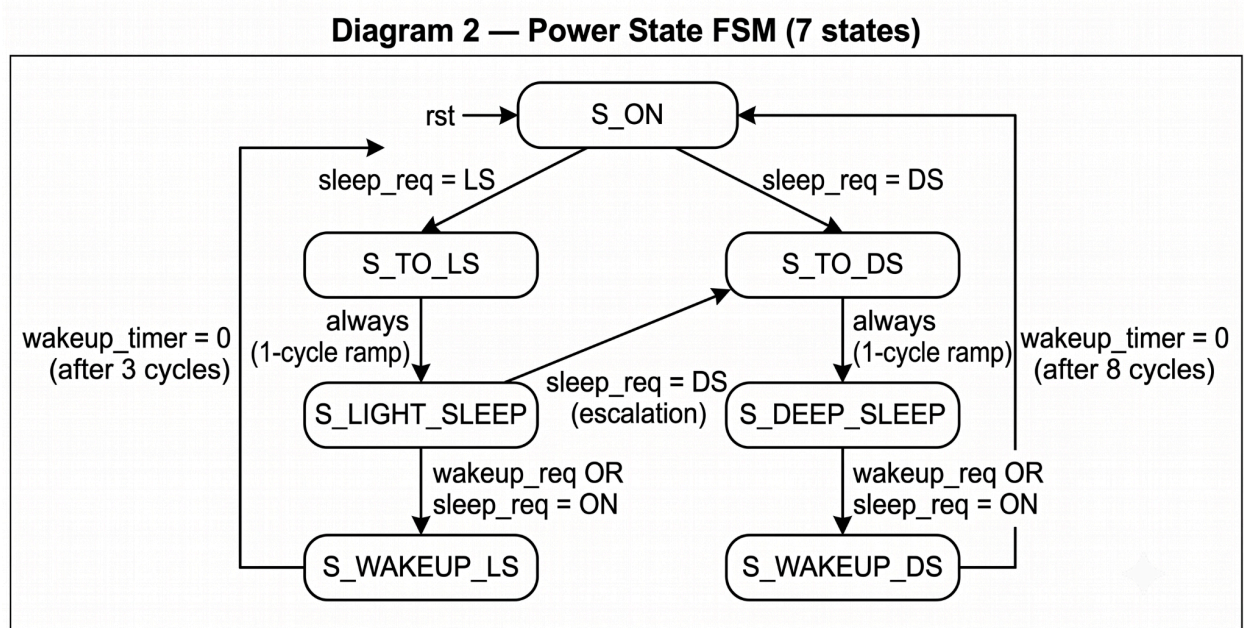


Fig. 1. Architecture of the adaptive multi-level power-gating controller.

The design is organized into two power domains: **PD_TOP** (always-on: activity monitor, hysteresis predictor, power state FSM, sleep controller) and **PD_ALU** (power-gated: the 32-bit ALU and its MTCMOS footer switch). The activity signal from the ALU feeds back into the always-on domain, which closes the control loop and decides when to gate **PD_ALU**. See diagram caption for full signal-level detail.

4. State Machine Behavior



Self-transitions (remaining in the same state) omitted for clarity.

Fig. 2. Power state FSM (7 states).

`power_state_fsm.v` implements 7 states: `S_ON`, `S_TO_LS`, `S_LIGHT_SLEEP`, `S_TO_DS`, `S_DEEP_SLEEP`, `S_WAKEUP_LS`, `S_WAKEUP_DS`. Wake-up latency is fixed at 3 cycles from Light Sleep and 8 cycles from Deep Sleep (values taken from the Agarwal et al. reference). Isolation (`iso_en`) is asserted one cycle before `sleep_n` deasserts on every sleep entry, and remains asserted through the entire wake-up sequence — this ordering was verified directly in the simulation log (see Section 6).

5. Functional Verification Status

Result: PASS. The design was re-simulated from a clean build (`rm -rf simv simv.daidir csruccli.key` followed by `bash scripts/run_sim_vcs.sh`) on June 28. All 5 testbench workloads completed without error:

Metric	Result
Total cycles simulated	562
Cycles in ON / Light Sleep / Deep Sleep	327 / 21 / 214
Valid results issued	141
Wake-up events	11
Mode transitions	34
Leakage saving (behavioral model)	14.4%

Workload 1 (burst traffic) correctly remained in ON throughout, confirming the hysteresis predictor's adaptive widening prevents premature sleep entry as designed. Workload 2 (sparse traffic) correctly progressed ON → Light Sleep → Deep Sleep. Workload 3 confirmed correct external wake-up behavior from Deep Sleep.

Important note on reproducibility: an earlier run of this same testbench against this same RTL produced a contradictory result (a spurious **WARN** during Workload 1 and a watchdog timeout in Workload 5). Root cause: VCS reused a stale compiled **simv** binary instead of recompiling against the current RTL ("the design hasn't changed and need not be recompiled" — incorrect, because the *logged artifacts* had not been regenerated after a prior RTL edit). This was resolved by force-deleting all VCS build artifacts before rerunning, and the repository's **.gitignore** was updated to stop tracking these regenerable files going forward, so this cannot recur silently.

6. Synthesis and Physical Implementation Status

scripts/dc_synthesis.tcl and **scripts/icc_place_route.tcl** exist and are structurally complete (two design points — low-power 15 ns and high-speed 8 ns clock constraints — plus a full ICC floorplan/CTS/route flow). However, neither script has been run to completion against a real or generic technology library — **dc_synthesis.tcl** currently points **TARGET_LIB** at a placeholder **typical.db**, and **icc_place_route.tcl**'s Milkyway/tech-file paths are unfilled placeholders (**/path/to/...**). **load_upf** is commented out in both scripts, meaning the power intent defined in **adaptive_pg.upf** has not yet been used to drive UPF-aware synthesis or P&R. No area, timing, or power numbers from DC/ICC currently exist — the only "power" numbers available so far (14.4% leakage saving) come from the testbench's behavioral leakage-weight model, not from a synthesized gate-level netlist.

7. Known Gaps Going Into the Research Plan

1. No baseline RTL exists (addressed in Deliverable 1's Baseline Definition; baseline RTL itself is pending).
2. No completed synthesis or P&R run — PPA numbers reported so far are simulation-level estimates, not post-synthesis measurements.
3. UPF is authored but not yet exercised through **load_upf** in either DC or ICC.
4. No multi-corner (voltage/PVT) analysis has been attempted yet — only nominal-corner simulation.

8. Audit Conclusion

The adaptive controller's RTL is functionally correct and its claimed behavior (adaptive thrashing suppression, 14.4% behavioral leakage saving) is reproducible from a clean build. The gap between this audit and a publishable result is entirely in the synthesis/P&R/baseline layer, not in RTL correctness — which sets the agenda directly for Steps 2.5 through 5.